

Data Analyst Agent 项目介绍

1. 项目概述

Data Analyst Agent 是一个面向多业务、多表数据场景的智能数据分析工作台。项目目标不是做一个固定报表或单一数据集看板，而是构建一个能够接入业务数据库、理解 Schema、推理表关联、自动生成 SQL、执行查询、自修复错误并输出分析结论的通用数据分析 Agent。

在传统数据分析流程中，业务人员往往需要先理解数据表结构、确认字段口径、寻找表之间的关联关系，再由数据分析师或开发人员编写 SQL。这个过程对业务人员门槛较高，对数据团队也会造成大量重复问数需求。本项目希望通过 AI Agent 将“自然语言问题”转化为“可追溯、可解释、可复盘的数据分析任务”，提升复杂业务数据的分析效率。

当前 demo 以电商业务为演示场景，基于开源订单数据合成了一个 Docker MySQL 多表数据库，包含用户、商品、订单、曝光、点击 5 张表。系统支持围绕年龄、性别、品类、支付方式、曝光点击购买转化等主题进行多表分析，适合展示“多表关联推理”和“自然语言到 SQL”的完整闭环。

2. 核心价值

2.1 降低数据理解门槛

系统会自动读取数据库表结构，生成字段画像，包括字段类型、样例值、缺失率、唯一值数量和业务含义猜测。用户不需要先手动翻表结构，也可以了解当前数据源中有哪些表、字段和可分析主题。

2.2 自动推理表关联关系

面对多表数据时，Agent 会识别 `customer_id`、`product_id`、`order_id` 等同名主外键候选字段，形成 Join Path。前端会将 Schema 证据和关联路径展示出来，让评审能够看到 SQL 不是“黑盒生成”，而是基于可验证的数据结构推理。

2.3 自然语言生成只读 SQL

用户输入自然语言问题后，Agent 会结合 Schema Profile、候选关联关系和业务词典生成 SQLite 兼容 SQL。SQL 生成阶段有安全约束，只允许 `SELECT` / `WITH` 查询，禁止写入、删除、建表等危险操作，并自动限制返回行数。

2.4 SQL 错误自修复

如果首次 SQL 执行失败，系统会捕获错误信息，并将原 SQL、错误原因和 Schema 重新交给模型进行修复。修复过程会沉淀在任务时间线中，前端可展示执行失败、修复动作和重试结果，体现 Agent 的闭环能力。

2.5 结果解释与深度分析

查询成功后，Agent 会基于 SQL 结果生成结构化图表建议和业务分析结论，包括关键发现、分层差异、可能原因和下一步分析建议。分析阶段被约束为只能基于当前 Schema 和查询结果说话，避免编造不存

在的退款、具体城市等字段。

2.6 支持多轮追问

系统支持在同一个任务中继续追问。后端会保留上一轮问题、SQL 和分析结论，并将用户追问改写为可独立理解的问题，从而支持“继续看女性用户”“那年轻用户里哪个品类最高”等多轮分析场景。

3. 系统架构

项目采用前后端分离架构：

- 前端：`Next.js + React + TypeScript`
- 后端：`FastAPI + LangChain + pandas + SQLAlchemy`
- 数据库：`Docker MySQL 8.0`
- 查询执行：当前 demo 将数据源统一加载为运行时 SQLite 查询连接，便于本地演示和只读安全控制
- 模型接入：OpenAI-compatible Chat Completions API，支持 OpenAI、DeepSeek、通义千问、OpenRouter 等兼容接口

核心链路如下：

1. 用户在前端输入自然语言分析问题。
2. 前端调用 `POST /api/tasks` 创建分析任务。
3. 后端为任务生成 AI 摘要标题，而不是直接使用用户问题。
4. 前端通过 SSE 调用 `/api/tasks/{task_id}/stream` 启动分析。
5. 后端加载 MySQL 数据源并生成 Schema Profile。
6. LangChain Agent 依次执行 Schema 理解、表关联推理、SQL 生成、SQL 执行、自修复、图表建议和深度分析。
7. 后端通过 SSE 将 `step`、`token`、`result`、`done` 事件实时推送给前端。
8. 前端展示任务时间线、Schema 证据、SQL、结果表、图表和洞察结论。

已有详细架构图可参考：

- [backend_architecture.md](#)
- [agent_backend_architecture_fireworks.svg](#)

4. 数据库设计

为了演示多表关联推理，项目基于原始 `淘宝用户行为.csv` 构造了一个电商分析数据库，库名为 `data_agent_demo`，通过 Docker MySQL 启动。

数据库包含 5 张表：

表名	类型	说明
<code>dim_customers</code>	用户维表	用户 ID、性别、年龄、年龄段、城市层级、会员等级
<code>dim_products</code>	商品维表	商品 ID、品类、商品名、品牌、标价、价格带、商品热度
<code>fact_orders</code>	订单事实表	订单 ID、用户 ID、商品 ID、品类、数量、单价、订单金额、支付方式、订单日期
<code>fact_product_exposures</code>	曝光事实表	曝光事件、用户、商品、日期、渠道、设备、是否购买链路
<code>fact_product_clicks</code>	点击事实表	点击事件、曝光 ID、用户、商品、日期、渠道、设备

当前 demo 数据规模：

- 用户表：约 99,457 行
- 商品表：10,000 行
- 订单表：约 99,457 行
- 曝光表：约 351,597 行
- 点击表：约 103,441 行

这套数据能够支撑以下典型问题：

- 按年龄段和性别统计订单数、销售额、客单价
- 不同年龄和性别用户从曝光到点击再到购买的转化率
- 年轻女性偏好的商品品类，以及与男性用户的差异
- 哪些品类高曝光、高点击但购买转化低
- 按性别和年龄段分析支付方式偏好

5. Agent 能力链路

5.1 Schema 理解

`agent/schema.py` 会对所有表生成结构化画像：

- 表名和行数
- 字段类型
- pandas 类型和 SQLite 类型
- 缺失率
- 唯一值数量
- 样例值
- 业务含义猜测
- 候选表关联关系

这些信息会作为 LLM 生成 SQL 的上下文，也会展示在前端 Schema 详情中。

5.2 表关联推理

系统会根据字段名和样例值重合度推断关系。例如：

- `fact_orders.customer_id -> dim_customers.customer_id`
- `fact_orders.product_id -> dim_products.product_id`
- `fact_product_clicks.exposure_id -> fact_product_exposures.exposure_id`

这使得 Agent 可以生成包含 Join 的多表 SQL，而不是只能分析单表。

5.3 SQL 生成

`agent/langchain_agent.py` 中的 `sql_generation_tool` 会把用户问题、Schema Profile、关联路径和多轮历史上下文发给大模型，要求模型只返回一条只读 SQL。

系统提示词明确约束：

- 只能输出 `SELECT` 或 `WITH`
- 只能使用 Schema 中真实存在的表和字段
- 不允许编造不存在字段
- 不允许执行写操作或数据库管理操作
- 缺少字段时返回 `schema_notice`，说明当前数据无法回答该问题

5.4 SQL 安全执行

SQL 执行前会经过安全校验：

- 禁止 `INSERT`、`UPDATE`、`DELETE`、`DROP`、`ALTER`、`CREATE` 等关键字
- 禁止多语句执行
- 自动追加 `LIMIT`
- 在只读沙箱中执行

5.5 错误自修复

如果 SQL 执行失败，Agent 会进入自修复流程：

1. 捕获执行错误。
2. 记录失败 SQL。
3. 结合 Schema 和错误信息重新请求模型修复。
4. 最多重试 2 次。
5. 将修复过程展示在前端任务时间线中。

5.6 图表建议

Agent 会根据查询结果生成结构化图表建议：

- `bar`

- line
- pie
- scatter
- bubble
- table

前端根据建议渲染 ECharts 图表。如果结果更适合表格，前端会展示表格提示，而不是强行绘图。

5.7 深度分析

深度分析阶段会生成 3-5 条中文业务洞察，包含：

- 关键发现
- 分层差异
- 异常点
- 可能原因
- 下一步分析建议

为了保证可信度，分析阶段会同时传入 SQL、结果字段、结果预览和 Schema 字段，并要求模型不要讨论不存在的指标或维度。

6. 前端工作台设计

前端采用“左侧 Agent 对话 + 右侧结构化分析工作区”的 Hybrid 形态。

6.1 任务首页

用户可以在首页输入自然语言问题，系统创建一个分析任务。任务标题由 AI 总结生成，例如：

- 用户问题：按年龄段和性别统计订单数、销售额、客单价
- AI 标题：年龄性别订单价值分析

6.2 分析工作台

工作台展示完整分析过程：

- Agent 对话
- 证据时间线
- Schema 证据
- Join Path
- SQL 生成与执行状态
- SQL 自修复记录
- 结果表
- 图表与洞察
- 审计信息

6.3 分享页

分享页用于展示完成后的分析任务，适合比赛录屏和评审查看。页面会保留核心 SQL、结果、图表和结论，使分析过程可以复盘。

7. 后端 API

后端主要接口如下：

接口	说明
GET /api/health	后端健康检查和模型配置状态
GET /api/data-sources	返回当前默认数据源
GET /api/data-sources/default/schema	返回默认数据源 Schema
GET /api/glossary	返回 demo 业务词典
POST /api/tasks	创建分析任务
GET /api/tasks	获取历史任务
GET /api/tasks/{task_id}	获取任务详情
GET /api/tasks/{task_id}/stream	第一轮分析 SSE
POST /api/tasks/{task_id}/runs/stream	多轮追问 SSE

SSE 事件类型：

- task：任务状态更新
- step：分析阶段更新
- token：深度分析流式文本
- result：最终结果
- error：可恢复错误
- done：本轮结束

8. 运行方式

8.1 启动 MySQL

```
docker compose -f docker-compose.mysql.yml up -d
```

8.2 构造 demo 数据

```
conda run -n data-agent-demo python scripts/bootstrap_mysql_demo.py --replace
```

8.3 启动后端

```
conda run -n data-agent-demo python -m uvicorn backend_app:app --host 127.0.0.1 --port 8001
```

8.4 启动前端

```
cd Data-Analyst-Agent-main  
npm run dev
```

前端默认访问后端地址：

```
http://127.0.0.1:8001
```

9. 环境变量

模型通过 OpenAI-compatible API 接入：

```
LLM_BASE_URL=https://openrouter.ai/api/v1  
LLM_API_KEY=your_api_key_here  
LLM_MODEL=openai/gpt-5.4  
DEMO_DATABASE_URL=mysql+pymysql://agent:agent_demo_123@127.0.0.1:3307/data_agent_demo
```

如果没有配置模型 API Key，后端不会崩溃，会返回可恢复错误提示。

10. 录屏讲解建议

你的视频录屏可以按以下顺序讲：

1. 项目定位：这是一个面向任意业务数据的数据分析 Agent，不是固定电商看板。
2. 数据源：展示 Docker MySQL 中的 5 张表，以及数据源页面识别出的表数量和字段数量。
3. Schema 理解：展示字段画像、样例值、业务含义和 Join Path。
4. 创建任务：输入“按年龄段和性别统计订单数、销售额、客单价”。
5. 实时执行：展示 SSE 时间线，说明 Agent 正在经历 Schema 理解、表关联推理、SQL 生成和 SQL 执行。
6. SQL 展示：打开 SQL 标签页，说明系统生成了只读 SQL，并带有安全限制。
7. 结果展示：展示结果表和图表。
8. 深度分析：展示 Agent 给出的关键发现和业务建议。
9. 多轮追问：继续问“女性用户里哪个年龄段客单价最高”，展示上下文追问能力。
10. 可信边界：可以演示问“退款率”时，系统会提示当前 Schema 没有退款字段，而不是编造数据。

11. 项目创新点

11.1 从“聊天机器人”升级为“分析任务”

项目不是简单把自然语言发给模型，而是把一次分析沉淀成任务对象，包含问题、标题、步骤、Schema 证据、SQL、执行结果、图表、洞察、审计和多轮追问。这样更适合企业数据分析场景中的协作和复盘。

11.2 Schema-first 的 SQL Agent

Agent 在生成 SQL 前会先理解 Schema，并将字段画像和 Join Path 作为上下文输入模型。相比直接让模型写 SQL，这种方式能更好约束模型使用真实字段，降低幻觉。

11.3 可解释的多表关联推理

系统不仅生成 Join SQL，还会把候选关联路径展示出来，让用户知道为什么这些表可以关联。这一点对复杂业务数据尤其重要。

11.4 SQL 自修复闭环

Agent 不仅会生成 SQL，还会执行 SQL、捕获错误、修复 SQL 并重试，形成从自然语言到结果的自动闭环。

11.5 可信边界控制

系统增加了缺失字段拦截和结果边界约束。例如没有退款字段就不分析退款率，没有具体城市字段就不编造用户城市。这让 demo 更接近真实企业数据分析系统的可靠性要求。

11.6 前后端实时流式联动

后端通过 SSE 持续推送 Agent 执行状态和分析 token，前端可以实时展示执行过程，而不是等待一个黑盒结果。这有利于提升用户对 Agent 的信任感。

12. 当前边界与后续规划

当前版本优先服务比赛 demo，已经具备完整可演示闭环，但仍有一些后续可扩展方向：

- 将任务状态从内存迁移到数据库，支持持久化历史任务。
- SQL 执行从运行时 SQLite 升级为直接在 MySQL/PostgreSQL 等数据库上执行只读查询。
- 增加正式的数据源管理页面，支持用户在前端新增数据库连接。
- 增加字段级权限、脱敏策略和 SQL 审计日志。
- 增加更复杂的业务词典与指标口径管理。
- 支持导出分析报告为 PDF 或 PPT。
- 支持更多图表类型和图表配置编辑。

13. 一句话总结

Data Analyst Agent 将自然语言问数、Schema 理解、多表 Join 推理、SQL 生成、自修复、图表展示和业务洞察整合成一个可解释、可复盘、可扩展的智能数据分析工作台，目标是在复杂业务数据场景中显著降低分析门槛并提升问数效率。